

Projet UML: Airwatcher

I. Architecture logicielle	2
I.1. Structure en trois couches	2
A. Couche Présentation (Interface Utilisateur) :	2
B. Couche Logique Métier (Application & Services) :	2
C. Couche d'Accès aux Données (Persistence) :	2
I.2. Justification du choix	2
II. Diagramme de classes	3
III. Diagrammes de séquence	4
IV. Description et pseudo-code des principaux algorithmes	5
IV.1. Algorithme 1 : Calcul de la qualité de l'air en un point précis	5
IV.2. Algorithme 2 : Calcul de la qualité moyenne de l'air dans une zone circulaire	6
IV.3. Algorithme 3 : Classement des capteurs par similarité	7
IV.4. Algorithme 4 : Détection de fraude (Capteurs non fiables)	8
IV.5. Algorithme 5 : Analyse de l'impact d'un purificateur d'air	9

I. Architecture logicielle

Pour répondre aux exigences de modularité, de performance et de sécurité, nous avons adopté une architecture multicouche (3-Tiers). Ce découpage permet de séparer les responsabilités et de faciliter l'évolution du système.

I.1. Structure en trois couches

A. Couche Présentation (Interface Utilisateur) :

- Implémente l'interface console (CLI) adaptée aux privilèges de chaque rôle (Agence, Fournisseur, Particulier).
- Gère l'interaction utilisateur et l'affichage des résultats sans contenir de logique métier.

B. Couche Logique Métier (Application & Services) :

- Centralise les algorithmes : calcul de l'indice ATMO , détection de fraudes , statistiques de zone et classement par similarité.
- Intègre la mesure de performance des traitements en millisecondes.
- Applique les règles de gestion (ex: exclusion des données non fiables).

C. Couche d'Accès aux Données (Persistence) :

- Assure la lecture et l'écriture des fichiers CSV (`sensors.csv`, `measurements.csv`, etc.).
- Abstrait la structure des fichiers pour fournir des objets exploitables par la couche métier.

I.2. Justification du choix

- **Sécurité** : Garantit que les utilisateurs n'ont aucun accès direct aux fichiers de données.
- **Maintenabilité** : Permet de modifier le format de stockage ou l'interface utilisateur sans impacter les algorithmes de calcul.
- **Testabilité** : Facilite la réalisation de tests unitaires sur les fonctionnalités critiques (moyennes, rankings) indépendamment de l'interface.

II. Diagramme de classes

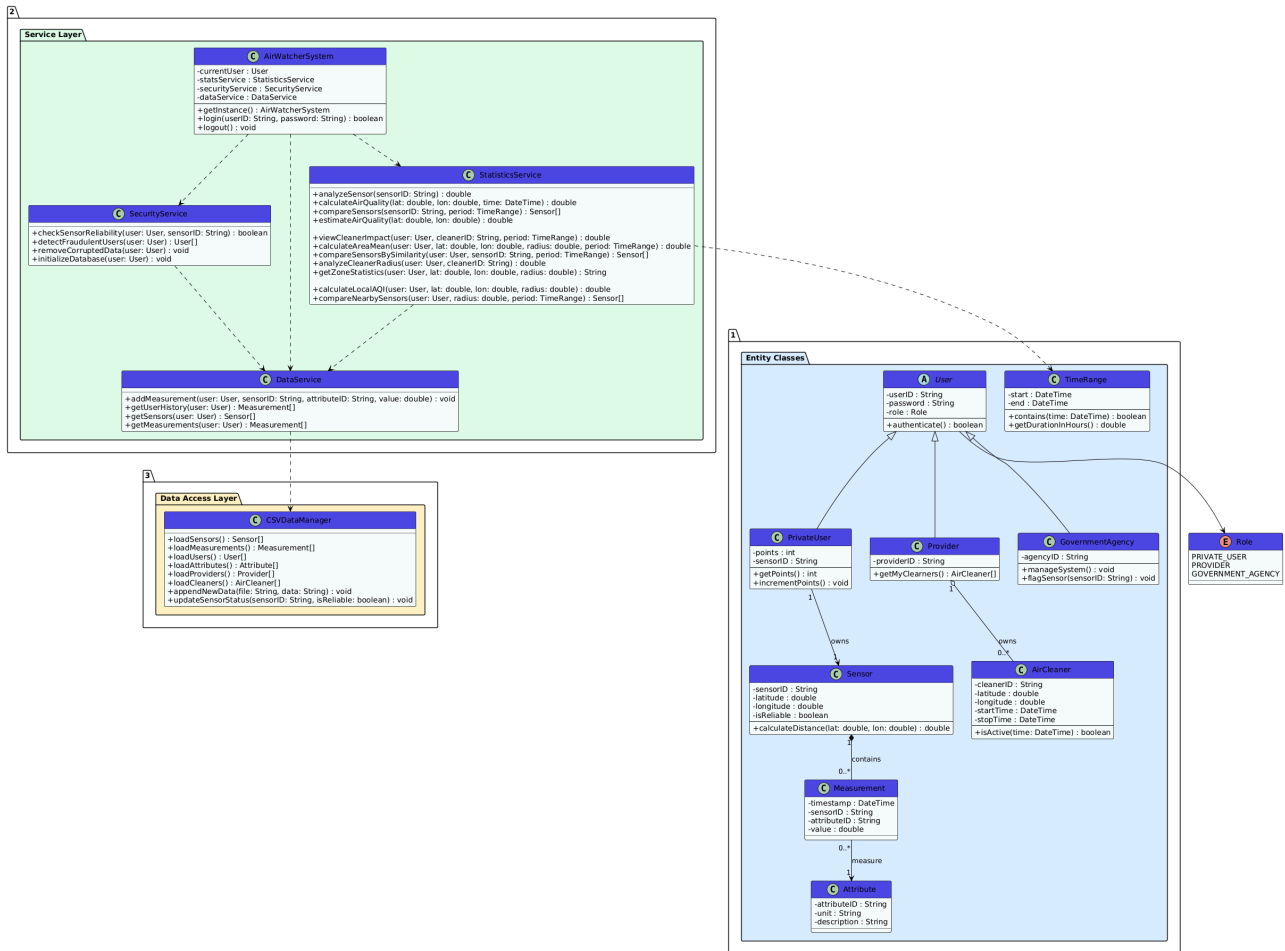


Fig. 1 : Diagramme de classes

III. Diagrammes de séquence

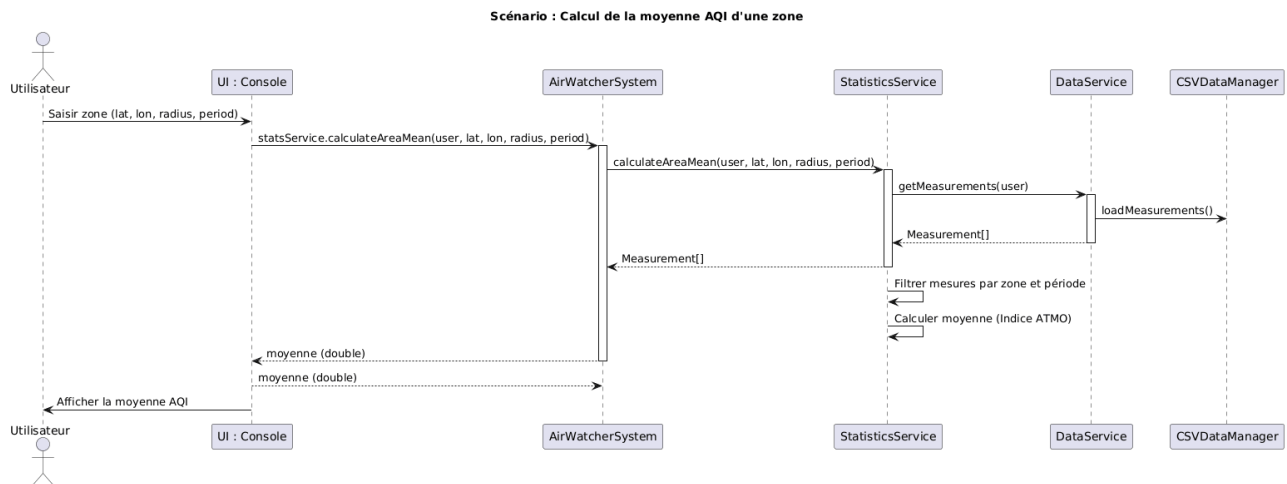


Fig. 2 : Diagramme de séquence pour le calcul de la moyenne AQI dans une zone

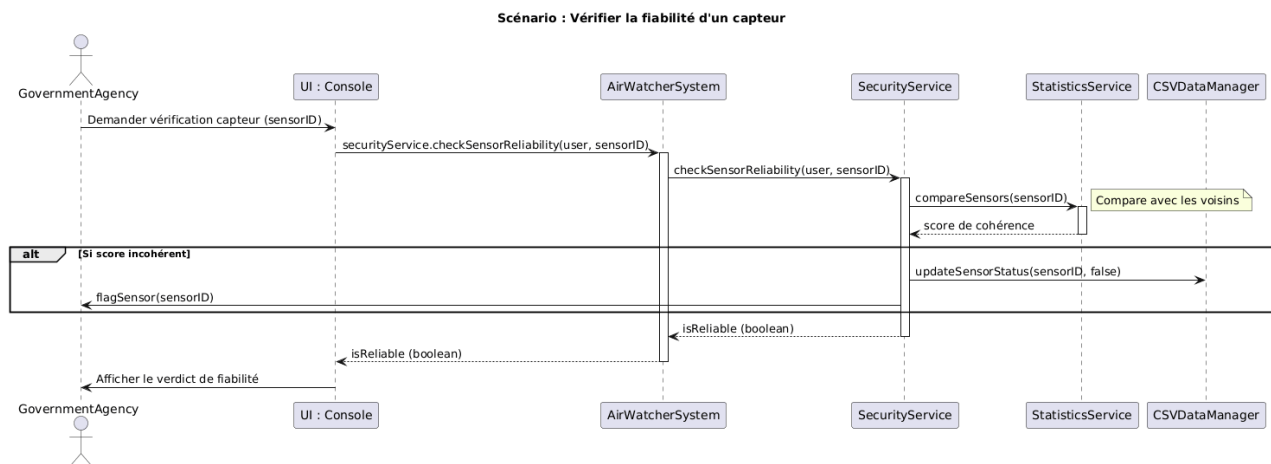


Fig. 3 : Diagramme de séquence pour vérifier la fiabilité d'un capteur

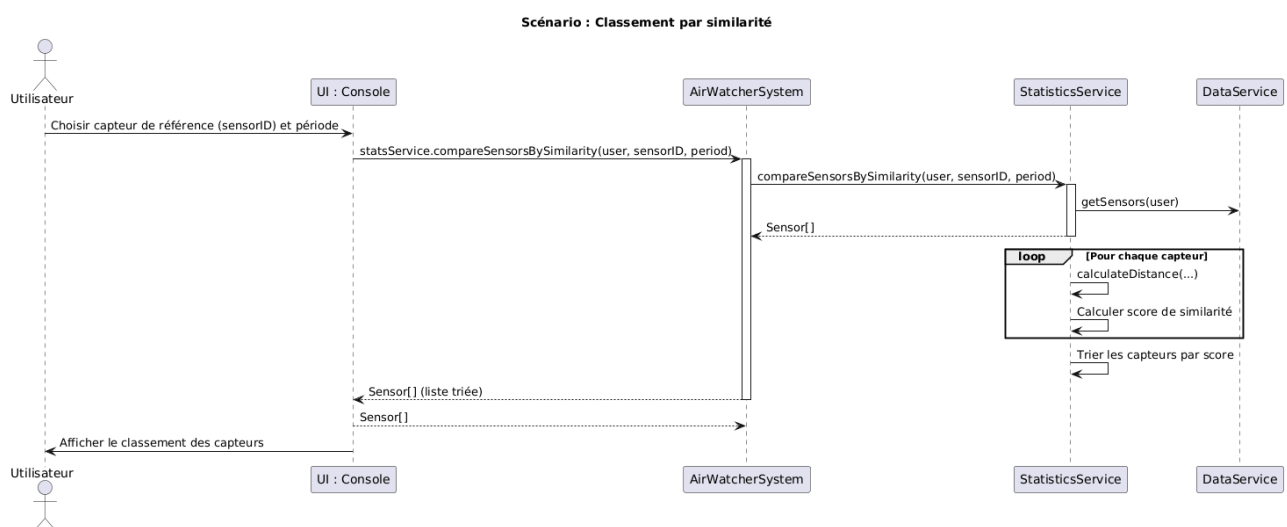


Fig. 4 : Diagramme de séquence pour la classement des capteurs par similarité

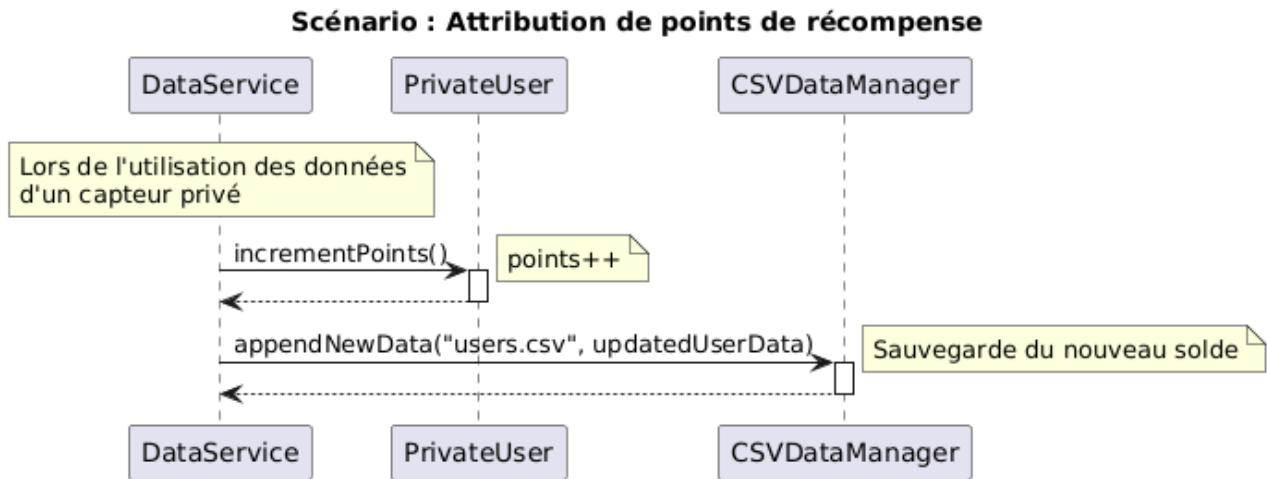


Fig. 5 : Diagramme de séquence pour l'attribution des points

IV. Description et pseudo-code des principaux algorithmes

Cette section détaille la logique interne des algorithmes les plus critiques du système AirWatcher. Pour répondre aux exigences non fonctionnelles de performance, chaque algorithme intègre un mécanisme de chronométrage mesurant son temps d'exécution en millisecondes. Les signatures et types de retour respectent strictement le diagramme de classes.

IV.1. Algorithme 1 : Calcul de la qualité de l'air en un point précis

- **Objectif** : Estimer l'indice de qualité de l'air (AQI) à une position géographique exacte où aucun capteur n'est forcément présent, en utilisant une interpolation spatiale des capteurs environnants.
- **Classe et Méthode** : `StatisticsService.calculateAirQuality(user: User, lat: double, lon: double, timestamp: DateTime) : double`
- **Logique de l'algorithme** :
 1. Récupération des capteurs fiables à proximité (via `calculateDistance`).
 2. Application d'une pondération par l'inverse de la distance ($\frac{1}{d^2}$) pour donner plus de poids aux capteurs les plus proches.
 3. Agrégation pondérée des valeurs des 4 types d'attributs.
 4. Conversion des moyennes pondérées en indice ATMO global.

Pseudo-code :

```

FONCTION calculateAirQuality(user, lat, lon, timestamp):
    tempsDebut = obtenirTempsActuel()

    tousCapteurs = DataService.getSensors(user)
    toutesMesures = DataService.getMeasurements(user)

    sommesPonderees = DICTIONNAIRE_VIDE // {attributeID: somme_valeurs_pondérées}
    sommePoids = DICTIONNAIRE_VIDE // {attributeID: somme_poids}

    // Étape 1 : Parcourir les capteurs pour l'interpolation
    
```

```
POUR CHAQUE capteur DANS tousCapteurs:
    SI capteur.isReliable == VRAI ALORS:
        distance = capteur.calculateDistance(lat, lon)

        // Éviter la division par zéro si on est pile sur un capteur
        poids = (distance < 0.1) ? 100 : 1 / (distance * distance)

        POUR CHAQUE mesure DANS toutesMesures:
            SI mesure.sensorID == capteur.sensorID ET mesure.timestamp ==
timestamp ALORS:
                sommesPonderees[mesure.attributeID] += (mesure.value * poids)
                sommePoids[mesure.attributeID] += poids

// Étape 2 : Calcul des moyennes pondérées par attribut
moyennesEstimees = DICTIONNAIRE_VIDE
POUR CHAQUE attrID DANS sommesPonderees:
    moyennesEstimees[attrID] = sommesPonderees[attrID] / sommePoids[attrID]

// Étape 3 : Conversion finale
indiceGlobal = convertirVersIndiceATM0(moyennesEstimees)

tempsFin = obtenirTempsActuel()
imprimer("Temps d'exécution : " + (tempsFin - tempsDebut) + " ms")

RETOURNER indiceGlobal
```

IV.2. Algorithme 2 : Calcul de la qualité moyenne de l'air dans une zone circulaire

- **Objectif :** Évaluer la qualité de l'air d'une zone géographique définie, durant une période spécifique.
- **Classe et Méthode :** `StatisticsService.calculateAreaMean(user: User, lat: double, lon: double, radius: double, period: TimeRange) : double`
- **Logique de l'algorithme :**
 1. Récupération de toutes les données via `DataService`.
 2. Filtrage spatial des capteurs utilisant `calculateDistance` et vérification de `isReliable`.
 3. Filtrage temporel des mesures utilisant `period.contains(timestamp)`.
 4. Calcul des moyennes et conversion en indice global (retourné sous forme de double).

Pseudo-code :

```
FONCTION calculateAreaMean(user, lat, lon, radius, period):
    tempsDebut = obtenirTempsActuel()

    // Accès aux données via DataService
    tousCapteurs = DataService.getSensors(user)
    toutesMesures = DataService.getMeasurements(user)

    capteursZone = LISTE_VIDE
```

```
// Étape 1 : Filtrage spatial et fiabilité
POUR CHAQUE capteur DANS tousCapteurs:
    distance = capteur.calculateDistance(lat, lon)
    SI distance <= radius ET capteur.isReliable == VRAI ALORS:
        AJOUTER capteur À capteursZone

// Étape 2 : Agrégation des mesures correspondantes
sommesAttributs = DICTIONNAIRE_VIDE
comptesAttributs = DICTIONNAIRE_VIDE

POUR CHAQUE capteur DANS capteursZone:
    POUR CHAQUE mesure DANS toutesMesures:
        // Vérification de l'appartenance et du TimeRange
        SI mesure.sensorID == capteur.sensorID ET
period.contains(mesure.timestamp) ALORS:
            sommesAttributs[mesure.attributeID] += mesure.value
            comptesAttributs[mesure.attributeID] += 1

// Étape 3 : Calcul des moyennes
moyennes = DICTIONNAIRE_VIDE
POUR CHAQUE attribut DANS sommesAttributs:
    moyennes[attribut] = sommesAttributs[attribut] / comptesAttributs[attribut]

indiceGlobal = convertirVersIndiceATM0(moyennes)

tempsFin = obtenirTempsActuel()
imprimer("Temps d'exécution : " + (tempsFin - tempsDebut) + " ms")

RETOURNER indiceGlobal // Type double selon le diagramme
```

IV.3. Algorithme 3 : Classement des capteurs par similarité

- **Objectif** : Identifier les zones ayant une qualité d'air similaire en comparant les données générées par tous les capteurs à celles d'un capteur de référence sur une période donnée.
- **Entrées** : targetSensorID (l'ID du capteur de référence), debut, fin.
- **Sorties** : Une liste de capteurs triée par ordre décroissant de similarité, avec le temps d'exécution.

Pseudo-code :

```
FONCTION compareSensorsBySimilarity(user, targetSensorID, period):
    tempsDebut = obtenirTempsActuel()

    tousCapteurs = DataService.getSensors(user)
    toutesMesures = DataService.getMeasurements(user)

    // Extraction des données de la cible pour la période
    donneesCible = LISTE_VIDE
    POUR CHAQUE mesure DANS toutesMesures:
        SI mesure.sensorID == targetSensorID ET period.contains(mesure.timestamp)
ALORS:
```

```
    AJOUTER mesure À donneesCible

scoresSimilarite = DICTIONNAIRE_VIDE

POUR CHAQUE capteur DANS tousCapteurs:
    SI capteur.sensorID == targetSensorID OU capteur.isReliable == FAUX ALORS:
        CONTINUER

    donneesAComparer = LISTE_VIDE
    POUR CHAQUE mesure DANS toutesMesures:
        SI mesure.sensorID == capteur.sensorID ET
period.contains(mesure.timestamp) ALORS:
            AJOUTER mesure À donneesAComparer

    differenceTotale = 0.0
    pointsCommuns = 0

    // Calcul de l'écart moyen
    POUR CHAQUE mesureCible DANS donneesCible:
        mesureCorrespondante = trouverMesure(donneesAComparer,
mesureCible.timestamp, mesureCible.attributeID)
        SI mesureCorrespondante EXISTE ALORS:
            differenceTotale += VALEUR_ABSOLUE(mesureCible.value -
mesureCorrespondante.value)
            pointsCommuns += 1

    SI pointsCommuns > 0 ALORS:
        ecartMoyen = differenceTotale / pointsCommuns
        scoresSimilarite[capteur] = ecartMoyen

listeTrie = TRIER_TABLEAU(scoresSimilarite, CROISSANT)

tempsFin = obtenirTempsActuel()
imprimer("Temps d'exécution : " + (tempsFin - tempsDebut) + " ms")

RETOURNER listeTrie // Retourne un tableau Sensor[]
```

IV.4. Algorithme 4 : Détection de fraude (Capteurs non fiables)

- **Objectif :** Analyser le comportement du capteur d'un utilisateur privé pour s'assurer qu'il ne falsifie pas les données.
- **Entrées :** sensorID (ID du capteur à analyser), debut, fin.
- **Sorties :** Un booléen (VRAI si frauduleux, FAUX si fiable).

Pseudo-code :

```
FONCTION checkSensorReliability(user, targetSensorID):
    tempsDebut = obtenirTempsActuel()

    tousCapteurs = DataService.getSensors(user)
    toutesMesures = DataService.getMeasurements(user)

    capteurAAanalyser = TROUVER_CAPTEUR(tousCapteurs, targetSensorID)
```

```
// Obtenir le voisinage via la méthode du capteur
capteursVoisinsFiables = LISTE_VIDE
POUR CHAQUE capteur DANS tousCapteurs:
    SI capteur.isReliable == VRAI ET capteur.sensorID != targetSensorID ALORS:
        SI capteur.calculateDistance(capteurAAAnalyser.latitude,
capteurAAAnalyser.longitude) <= 10.0 ALORS:
            AJOUTER capteur À capteursVoisinsFiables

anomaliesDetectees = 0
totalMesures = 0
SEUIL_TOLERANCE = 0.30
SEUIL_FRAUDE = 0.50

// Isoler les mesures du capteur cible
mesuresCible = EXTRAIRE_MESURES(toutesMesures, targetSensorID)

POUR CHAQUE mesure DANS mesuresCible:
    valeurReference = calculerMoyenneVoisinsAUnInstant(capteursVoisinsFiables,
toutesMesures, mesure.timestamp, mesure.attributeID)

    SI valeurReference EST_VALIDE ALORS:
        totalMesures += 1
        ecartAbsolu = VALEUR_ABSOLUE(mesure.value - valeurReference)

        SI ecartAbsolu > (valeurReference * SEUIL_TOLERANCE) ALORS:
            anomaliesDetectees += 1

estFiable = VRAI
SI totalMesures > 0 ET (anomaliesDetectees / totalMesures) > SEUIL_FRAUDE
ALORS:
    estFiable = FAUX
    capteurAAAnalyser.isReliable = FAUX
    // Mise à jour de la persistance via la couche de données (accès direct
DAO)
    CSVDataManager.updateSensorStatus(targetSensorID, FAUX)

tempsFin = obtenirTempsActuel()
imprimer("Temps d'exécution : " + (tempsFin - tempsDebut) + " ms")

RETOURNER estFiable // Retourne boolean (VRAI si fiable, FAUX si frauduleux)
```

IV.5. Algorithme 5 : Analyse de l'impact d'un purificateur d'air

- **Objectif** : Mesurer l'efficacité d'un purificateur (AirCleaner) en comparant la qualité de l'air dans sa zone d'effet avant et après sa mise en marche (ou avec/sans son activation).
- **Classe et Méthode** : StatisticsService.viewCleanerImpact(user: User, cleanerID: String, period: TimeRange) : double
- **Logique de l'algorithme** :

1. Localisation du purificateur et identification des capteurs situés dans son rayon d'action.
2. Division de la période (period) en deux segments : avant activation et après activation (basé sur le startTime du cleaner).
3. Calcul de la qualité moyenne de l'air pour ces deux segments de temps.
4. Calcul du différentiel (amélioration en pourcentage ou en points d'indice).

Pseudo-code :

```
FONCTION viewCleanerImpact(user, cleanerID, period):
    tempsDebut = obtenirTempsActuel()

    // 1. Récupérer les infos du cleaner
    cleaner = DataService.getCleanerById(cleanerID)
    rayonEffet = cleaner.radius

    // 2. Identifier les capteurs impactés
    tousCapteurs = DataService.getSensors(user)
    capteursImpactes = LISTE_VIDE
    POUR CHAQUE capteur DANS tousCapteurs:
        SI capteur.calculateDistance(cleaner.lat, cleaner.lon) <= rayonEffet ALORS:
            AJOUTER capteur.sensorID À capteursImpactes

    // 3. Séparer la période en "Avant" et "Après" activation
    // On suppose que le cleaner a un timestamp d'activation (cleaner.startTime)
    periodeAvant = period.getSubRangeBefore(cleaner.startTime)
    periodeApres = period.getSubRangeAfter(cleaner.startTime)

    // 4. Calculer la qualité moyenne pour chaque période (Utilise l'Algo 1)
    qualiteAvant = calculateAreaMean(user, cleaner.lat, cleaner.lon, rayonEffet,
    periodeAvant)
    qualiteApres = calculateAreaMean(user, cleaner.lat, cleaner.lon, rayonEffet,
    periodeApres)

    // 5. Calculer l'impact (différence)
    // Un impact positif signifie une baisse de la pollution (amélioration de
    l'indice)
    impactRelatif = qualiteAvant - qualiteApres

    tempsFin = obtenirTempsActuel()
    imprimer("Analyse effectuée en : " + (tempsFin - tempsDebut) + " ms")

    RETOURNER impactRelatif
```